



ALANYA ÜNİVERSİTESİ MESLEK YÜKSEKOKULU
BPR 251 –MOBİL UYGULAMA GELİŞTİRME DERSİ
FİNAL SINAVI PROJE ÖDEVİ RAPORU

Öğrenci Bilgileri	
Adı Soyadı:	Arda DURAN
Öğrenci Numarası:	240053017
Bölümü:	Bilgisayar Programcılığı
Sınıfı:	2. Sınıf
Proje Github Linki	https://github.com/ArdaDuran7/SanayiRehberim
Proje Başlığı	ALANYA ÜNİVERSİTESİ SANAYİ REHBERİM PROJESİ

Teslim Kontrolü:

- [X] Raporun pdf dosyası öğrenci tarafından teslim edildi.
- [X] Projenin kodlarını içeren dosyalar öğrenci tarafından teslim edildi.
- [X] Rapor uzunluğu kapak hariç 5-10 sayfa aralığındadır.

İmza: _____

SANAYİ REHBERİM PROJE RAPORU

1. UYGULAMA ÖZETİ VE PROBLEM TANIMI

1.1. Projenin Tanımı ve Amacı "Sanayi Rehberim", otomotiv sektöründeki hizmet sağlayıcılar (oto tamir, kaporta, elektrik, lastik vb.) ile hizmet alıcıları (araç sahipleri) dijital bir platformda buluşturan; entegre randevu, lokasyon ve değerlendirme yönetimi sunan mobil tabanlı bir yazılım projesidir. Projenin temel amacı, sanayi sitelerindeki geleneksel ve verimsiz iş yapış süreçlerini modernize ederek zaman tasarrufu sağlamak ve hizmet kalitesini artırmaktır.

1.2. Problem Tespiti Günümüzde araç sahipleri, araçlarında meydana gelen arızalarda veya periyodik bakım ihtiyaçlarında şu sorunlarla karşılaşmaktadır:

- **Zaman Kaybı:** Sanayi sitelerinde dükkan dükkan dolaşarak müsait ve yetkin usta arama zorunluluğu.
- **Bilgi Eksikliği:** Hizmet alınacak dükkanın uzmanlık alanı, çalışma saatleri veya daha önceki müşteri memnuniyet oranları hakkında şeffaf bilgiye ulaşılamaması.
- **Randevu Karmaşası:** Telefon trafiği veya sözlü iletişimle alınan randevuların unutulması, çakışması veya takibinin yapılamaması.

1.3. Çözüm ve Hedef Kitle Geliştirilen uygulama, bu problemleri şu yöntemlerle çözmektedir:

- **Kategorizasyon:** Kullanıcıların ihtiyacına yönelik (Motor, Kaporta vb.) spesifik filtreleme yapabilmesi.
- **Dijital Randevu:** Kullanıcının uygulama üzerinden tarih ve saat seçerek talep oluşturması ve dükkan sahibinin bu talebi yönetebilmesi.
- **Şeffaflık:** Dükkanların konum, görsel ve puan bilgilerinin açıkça sergilenmesi.

Hedef Kitle:

1. **Son Kullanıcılar (B2C):** Aracının bakımı için güvenilir usta arayan bireysel araç sahipleri.
2. **Hizmet Sağlayıcılar (B2B):** İşletmesini dijitalleştirmek, müşteri portföyünü genişletmek ve iş akışını mobilden yönetmek isteyen sanayi esnafı.

2. TEKNİK ALTYAPI VE MİMARİ YAKLAŞIM

Uygulama geliştirme sürecinde, Google tarafından geliştirilen ve tek kod tabanıyla çoklu platform (Android/iOS) desteği sunan **Flutter** framework'ü kullanılmıştır. Programlama dili olarak Nesne Yönelimli (OOP) yapısı güçlü olan **Dart** tercih edilmiştir.

2.1. Backend Mimarisi (Google Firebase)

Sunucu tarafında (Backend) maliyet etkinliği ve hızlı geliştirme imkanı sunan "Serverless" (Sunucusuz) mimari tercih edilmiş ve **Google Firebase** ekosistemi kullanılmıştır.

- **Authentication (Kimlik Doğrulama):** Kullanıcıların güvenli bir şekilde kayıt olması ve giriş yapması için Firebase Auth servisi kullanılmıştır. E-posta ve şifre tabanlı doğrulama yöntemi entegre edilmiştir.
- **Cloud Firestore (Veritabanı):** Uygulamanın veri katmanı için NoSQL tabanlı, ölçeklenebilir bir veritabanı olan Firestore tercih edilmiştir. İlişkisel veritabanlarının aksine, veriler "Koleksiyonlar" (Collections) ve "Dokümanlar" (Documents) hiyerarşisinde tutularak okuma/yazma hızında yüksek performans sağlanmıştır.

2.2. State Management (Durum Yönetimi) Tercihi

Projede Durum Yönetimi için **Hibrit Yaklaşım (setState + StreamBuilder)** benimsenmiştir.

- **Neden StreamBuilder?** Uygulamanın en kritik fonksiyonu olan "Randevu Takibi" ve "Dükkan Listeleme", gerçek zamanlı (Real-Time) veri akışını zorunlu kılmaktadır. StreamBuilder widget'ı, Firestore ile kurulan Stream bağlantısını sürekli dinler. Veritabanında bir değişiklik olduğunda (örneğin; bir usta randevuyu onayladığında), arayüz (UI) herhangi bir kullanıcı tetiklemesine gerek kalmadan kendini otomatik olarak günceller. Bu yapı, kullanıcı deneyimini (UX) en üst seviyeye çıkarmaktadır.
- **Neden setState?** Form alanlarındaki metin değişimleri, Loading animasyonlarının kontrolü veya sekmeler (Tabs) arası geçişler gibi "Ephemeral State" (Geçici Durum) olarak adlandırılan ve sadece o anki ekranı ilgilendiren durumlar için setState kullanılmıştır. Bu sayede Provider veya Bloc gibi harici kütüphanelerin getireceği kod karmaşasından kaçınılmış ve uygulamanın derleme boyutu düşük tutulmuştur.

2.3. Kullanılan Kütüphaneler (Dependencies)

pubspec.yaml dosyasında tanımlanan ve projeye yetenek kazandıran temel paketler şunlardır:

1. **firebase_core:** Firebase servislerini Flutter motoruna bağlayan köprü görevini görür.

2. **firebase_auth:** Oturum yönetimi, token takibi ve güvenli çıkış işlemleri için kullanılmıştır.
3. **cloud_firestore:** Veritabanına asenkron (async) sorgular atılması ve verilerin JSON benzeri formatta çekilmesi için kullanılmıştır.
4. **image_picker:** Kullanıcının cihazın galerisine erişerek profil fotoğrafı veya dükkan görseli yükleyebilmesi için entegre edilmiştir.
5. **url_launcher:** Dükkan detay sayfasındaki adres bilgisi üzerinden, harici navigasyon uygulamalarının (Google Maps) başlatılması için kullanılmıştır.
6. **intl:** Tarih (DateTime) nesnelerinin kullanıcı dostu formatlara (örn: "14.01.2026") dönüştürülmesi için kullanılmıştır.

3. KRİTİK KOD BLOKLARI

Projenin navigasyon güvenliğini ve kullanıcı akışını yöneten en temel yapı taşı **AuthGate** sınıfıdır. Bu sınıf, uygulamanın giriş noktasıdır ve bir "Karar Mekanizması" olarak çalışır.

İşlevsel Açıklama: Aşağıdaki kod bloğu; uygulama başlatıldığında (Cold Start) önce 3 saniyelik bir animasyon (Splash Screen) gösterir. Bu süre zarfında arka planda Firebase bağlantısını ve cihazda kayıtlı bir oturum olup olmadığını kontrol eder.

```
class AuthGate extends StatefulWidget {
  const AuthGate({super.key});
  @override
  State<AuthGate> createState() => _AuthGateState();
}

class _AuthGateState extends State<AuthGate> {
  bool _splashGosteriliyor = true;

  @override
  void initState() {
    super.initState();
    // 1. ZAMANLAYICI: 3 saniye sonra Splash ekranını kaldırır.
    Future.delayed(const Duration(seconds: 3), () {
      if (mounted) setState(() => _splashGosteriliyor = false);
    });
  }

  @override
  Widget build(BuildContext context) {
    // 2. GÖRÜNÜM KONTROLÜ: Eğer splash süresi bitmediyse Logo göster.
    if (_splashGosteriliyor) {
      return Scaffold( body: Container(...) ); // Logo Kodları
    }

    // 3. DİNAMİK YÖNLENDİRME (StreamBuilder):
    // FirebaseAuth.instance.authStateChanges() metodu, kullanıcının oturum
    // durumunu (Giriş/Çıkış) sürekli dinler.
    return StreamBuilder<User?>(
      stream: FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        // Eğer stream doluysa (snapshot.hasData), yani kullanıcı giriş yapmışsa:
        if (snapshot.hasData) return const MagazaListesiEkran();

        // Eğer stream boşsa, yani kullanıcı oturumu yoksa:
        return const GirişKayıtEkran();
      },
    );
  }
}
```

Neden Kritik? Bu yapı sayesinde:

1. **Oturum Sürekliliği:** Kullanıcı uygulamayı kapatıp açtığında tekrar şifre girmek zorunda kalmaz.
2. **Güvenlik:** Kullanıcı "Çıkış Yap" butonuna bastığında veya hesabı silindiğinde, Stream anında tetiklenir ve kullanıcıyı güvenlik gereği GirişKayıtEkranına atar. Bu, uygulamanın yetkisiz erişimlere karşı korunmasını sağlar.

4. TEST SENARYOLARI (SINIR ZORLAMA TESTLERİ)

Uygulamanın kararlılığını (stability) ölçmek için aşağıdaki "Edge Case" (Uç Durum) testleri uygulanmıştır:

Senaryo 1: Ağ Bağlantısı Kesintisi

- **Test:** Dükkan listesi yüklenirken cihazın internet bağlantısı kesildi.
- **Beklenen Davranış:** Uygulamanın çökmemesi, kullanıcıya hata mesajı göstermesi.
- **Gerçekleşen:** StreamBuilder widget'ının snapshot.hasError bloğu devreye girdi. Ekranda sonsuz yükleme ikonu yerine "Bir hata oluştu" mesajı gösterildi.

Senaryo 2: Veri Bütünlüğü ve Hesap Silme (Transaction Safety)

- **Test:** Kullanıcı profil ayarlarından hesabını sildiğinde, sadece oturum mu kapanıyor yoksa veritabanındaki kayıtlar da siliniyor mu?
- **Gerçekleşen:** Silme butonuna basıldığında önce bir onay penceresi (Dialog) açıldı. Onay verildiğinde kod sırasıyla; 1) Firestore users koleksiyonundaki dokümanı sildi, 2) Firebase Authentication kullanıcısını sildi. İşlem sonucunda AuthGate devreye girerek kullanıcıyı giriş ekranına yönlendirdi. Veri tabanında "Hayalet Kayıt" (Orphan Data) kalmadığı teyit edildi.

Senaryo 3: Hatalı Veri Girişi (Input Validation)

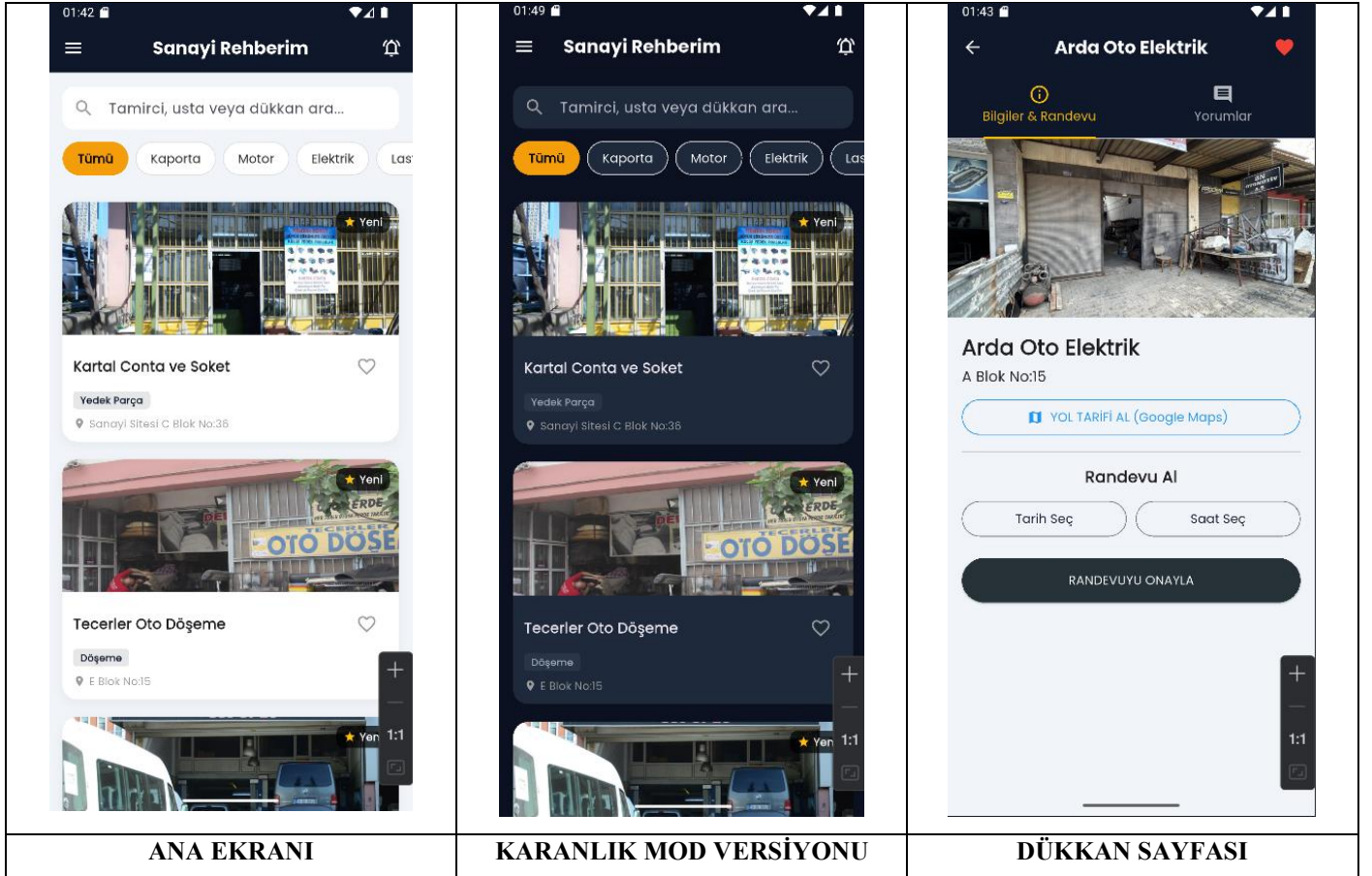
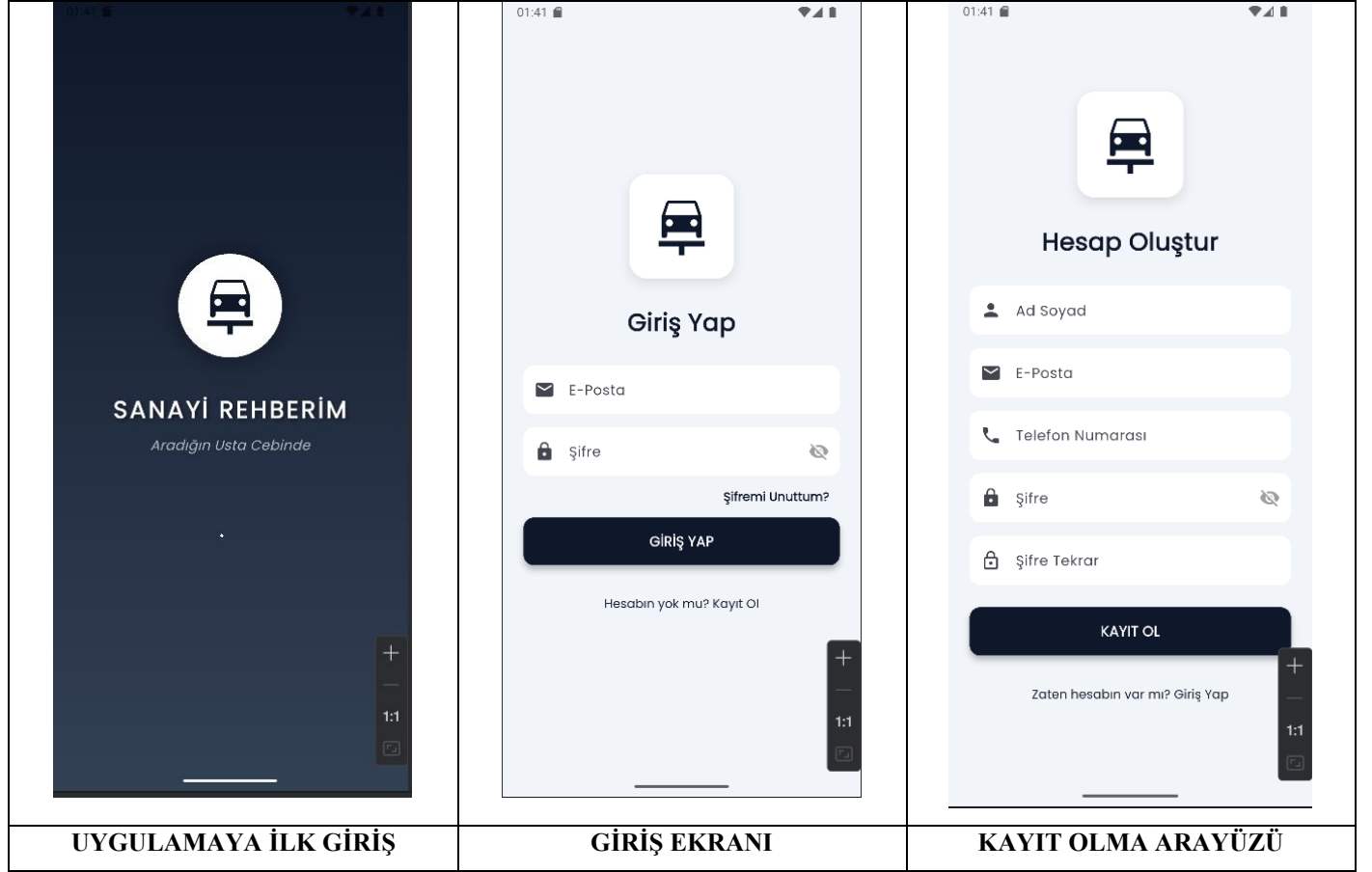
- **Test:** Randevu oluşturma ekranında Tarih ve Saat seçmeden "Onayla" butonuna basıldı.
- **Gerçekleşen:** Fonksiyonun başındaki if (secilenTarih == null) kontrolü devreye girdi ve işlem durduruldu. Kullanıcıya "Lütfen tarih seçiniz" uyarısı (SnackBar) verildi. Böylece veritabanına null (boş) veri gönderilmesi engellendi.

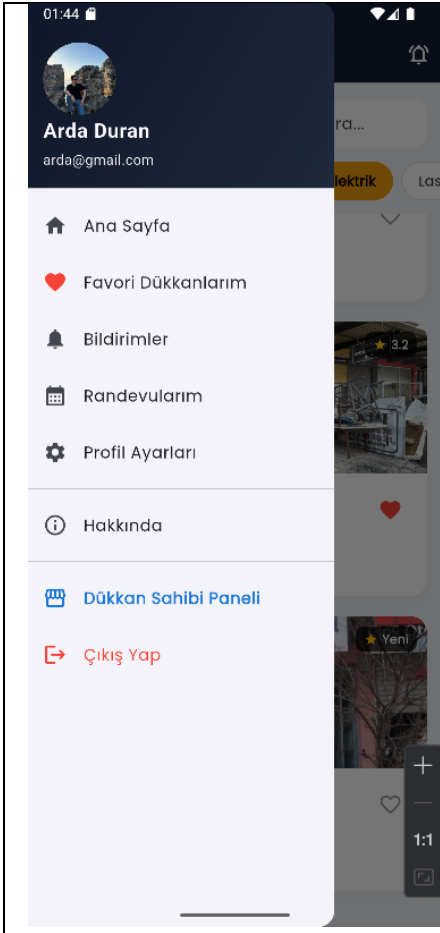
5. UYGULAMA AKIŞI (WORKFLOW)

Uygulamanın kullanıcı deneyimi (UX) akışı, doğrusal ve sezgisel bir yapıda kurgulanmıştır:

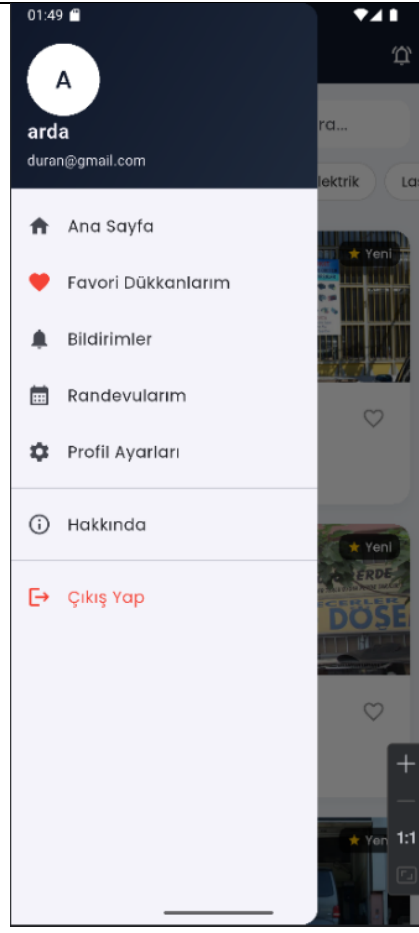
1. **Başlatma (Initialization):** Kullanıcı uygulamayı açar. main.dart dosyasında Firebase.initializeApp() çalışır. Başarılı olursa Splash ekranı görüntülenir.
2. **Kimlik Doğrulama:**
 - Kullanıcı yeni ise -> "Kayıt Ol" sekmesinden E-posta/Şifre ile kayıt olur. (Hata durumunda: "Bu mail kullanımda" uyarısı verilir).
 - Kullanıcı kayıtlı ise -> "Giriş Yap" sekmesinden sisteme dahil olur.
3. **Ana Sayfa (Keşfet):** Kullanıcı, veritabanından çekilen dükkan listesini görür. Üstteki yatay kaydırılabilir menüden (Chip Widget) "Kaporta", "Motor" gibi filtreleri seçerek listeyi daraltır.
4. **Detay ve Aksiyon:** İlgilendiği dükkana tıklar. Detay sayfasında dükkan görselini, adresini ve puanını görür. "Randevu Al" takvimini kullanarak tarih seçer ve talebini iletir.
5. **Takip (Randevularım):** Kullanıcı, oluşturduğu randevunun durumunu (Bekliyor/Onaylandı/Reddedildi) bu ekrandan renk kodlarıyla takip eder.
6. **Yönetim (Profil):** Kullanıcı profil fotoğrafını değiştirebilir, kişisel bilgilerini güncelleyebilir veya uygulamadan çıkış yapabilir.

6. MEVCUT DURUM

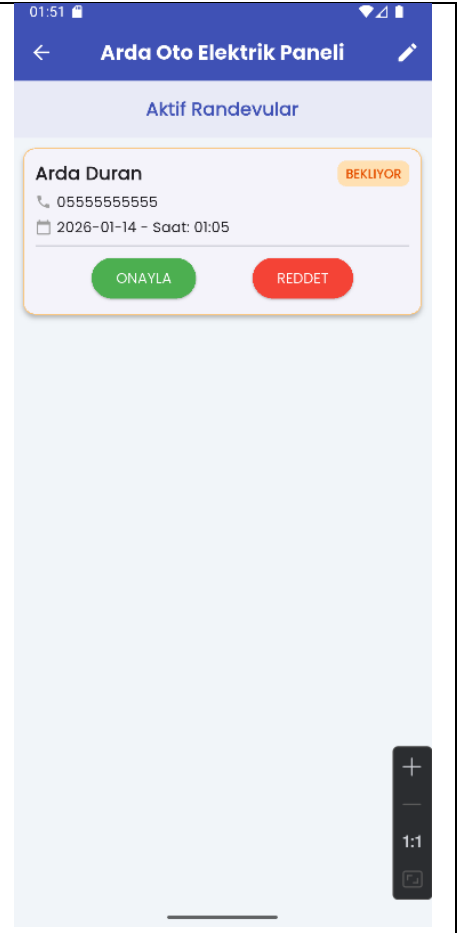




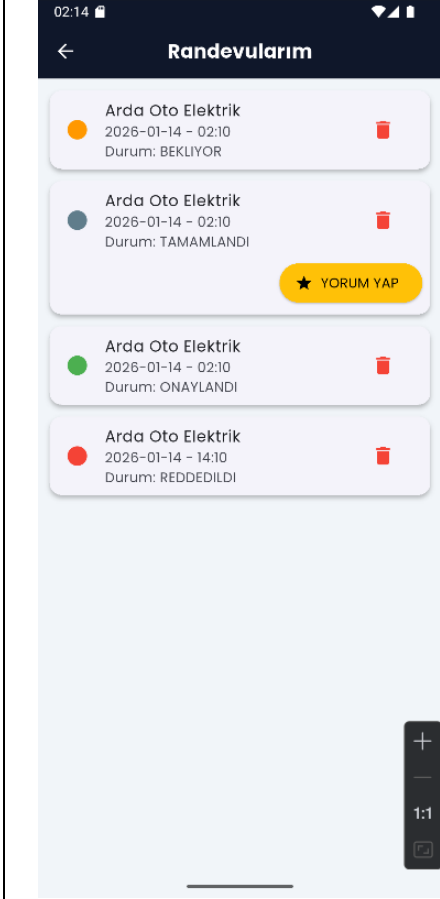
KURUMSAL KULLANICI MENÜ



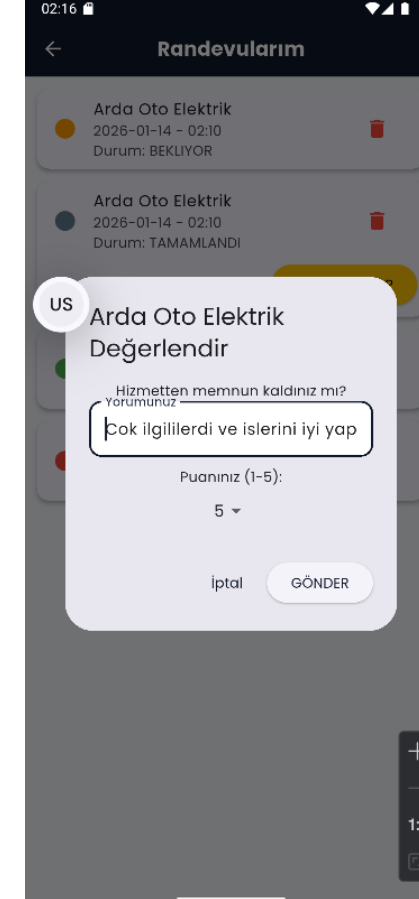
MÜŞTERİ KULLANICI MENÜ



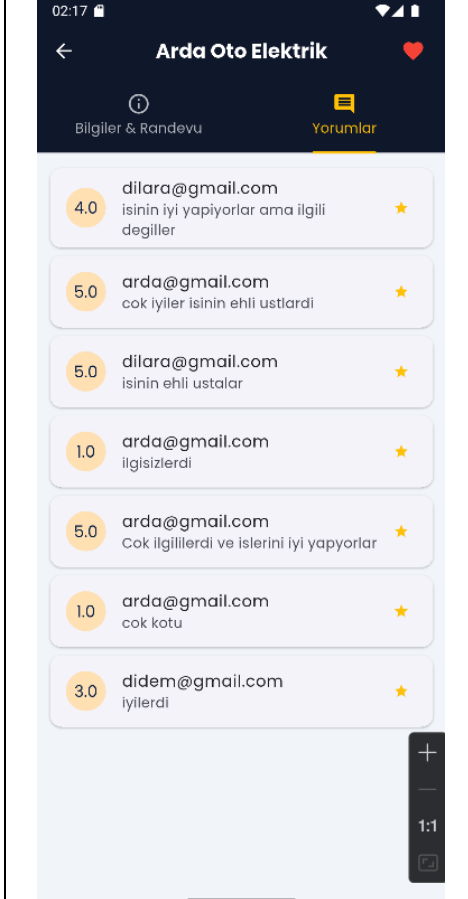
DÜKKAN PANELİ



MÜŞTERİ RANDEVU LİSTESİ



YORUM YAPMA KISMI



DÜKKAN YORUMLARI

7. KARŞILAŞILAN HATALAR VE ÇÖZÜMLERİ

Geliştirme sürecinde karşılaşılan majör teknik sorunlar ve uygulanan mühendislik çözümleri:

1. Async Gap ve Context Kaybı Hatası:

- **Problem:** Kayıt olma işlemi (await) tamamlandıktan sonra kullanıcıya başarı mesajı (SnackBar) göstermek istediğimde, Do not use BuildContexts across async gaps uyarısı alıyordum. Çünkü işlem bitene kadar kullanıcının ekranı değiştirme ihtimali vardı.
- **Çözüm:** Mesaj gösterme kodundan hemen önce if (mounted) kontrolü eklendi. Bu sayede, kodun çalıştığı "Widget"ın hala ekranda çizili olup olmadığı kontrol edilerek hata (Exception) oluşması engellendi.

2. Firebase Başlatma Yarış Durumu (Race Condition):

- **Problem:** Uygulamanın ilk açılışında beyaz ekranda takılma veya "Firebase has not been initialized" hatası alınıyordu. Flutter motoru, Firebase servislerinden daha hızlı yüklendiği için senkronizasyon sorunu yaşıyordu.
- **Çözüm:** main() fonksiyonu async hale getirildi. En başa WidgetsFlutterBinding.ensureInitialized(); komutu eklenerek Flutter motorunun hazır olması beklendi, ardından await Firebase.initializeApp(); ile Firebase'in tamamen yüklenmesi garanti altına alındıktan sonra runApp() çağrıldı.

3. Kullanıcı Deneyimi ve Hızlı Geçiş Sorunu:

- **Problem:** Kullanıcı kayıt olduğunda işlem çok hızlı gerçekleşiyor ve AuthGate kullanıcıyı anında ana sayfaya atıyordu. Bu yüzden başarı mesajı kullanıcı tarafından fark edilemiyordu.
- **Çözüm:** Basit bir metin mesajı yerine, sayfadan bağımsız (Overlay) olarak çalışan ve barrierDismissible: false özelliğine sahip bir AlertDialog tasarlandı. Bu pencerenin içine bir zamanlayıcı (Future.delayed) konularak, sayfa değişse bile mesajın 2 saniye boyunca ekranda kalması sağlandı.

8. SONUÇ VE GELECEK PERSPEKTİFİ

Bu proje ile, sanayi sitelerindeki fiziksel randevu alma ve usta bulma süreçleri başarıyla dijitalize edilmiştir. Geliştirilen uygulama; güvenli kimlik doğrulama altyapısı, gerçek zamanlı veritabanı senkronizasyonu ve kullanıcı dostu arayüzü ile Mobil Uygulama Geliştirme dersi kazanımlarını tam olarak yansıtmaktadır.

Gelecek Sürümler İçin Planlamalar:

- **Gelişmiş Admin Paneli:** Mevcut dükkan sahipleri için sunulan panelin, gelir-gider takibi ve personel yönetimi özellikleri ile zenginleştirilmesi.
- **Push Bildirimleri (FCM):** Şu anki sürümde uygulama içi çalışan bildirim sisteminin, uygulama kapalıyken de çalışacak şekilde "Firebase Cloud Functions" kullanılarak tam otomatik hale getirilmesi.
- **Yapay Zeka Destekli Chat:** Usta ve müşteri arasında, araçtaki sorunun fotoğrafının çekilip yapay zeka ile ön analizinin yapılabileceği bir sohbet modülü.